

tasks:

```
- name: Stop nginx server
  service:
    name: nginx
    state: stopped

- name: Remove existing directory
  shell: /bin/rm -rf /var/www/html/home/*

- name: Home directory should exist
  file:
    path: /var/www/html/home
    state: directory
    mode: 0755

- name: Copy latest files
  copy:
    src: ../../files/_site/
    dest: /var/www/html/home/
    directory_mode: yes

- name: Copy example.domain nginx file
  copy:
    src: ../../files/example.domain
    dest: /etc/nginx/sites-available/

- name: Create link file
  file:
    src: /etc/nginx/sites-available/example.domain
    dest: /etc/nginx/sites-enabled/example.domain
    owner: root
    group: root
    force: yes
    state: link

- name: Start nginx
  service:
    name: nginx
    state: started

- wait_for:
  port: 80
```

The Nginx Web server is first stopped, as we are changing the Web content. If you have multiple instances of the Web server running, you can run the playbook in a rolling upgrade fashion. In this way, if client requests occur during the upgrade, you will not have a downtime. In this playbook, we first remove the contents of the old directory, and then copy the new contents. Depending on the volume of content, this may or may not suit your requirements. You can also use the *synchronise* module in Ansible (http://docs.ansible.com/ansible/latest/modules/synchronize_module.html). This uses *rsync* to synchronise files between your

local host and the remote server.

The *example.domain* is the Nginx configuration file for the website. As per the Nginx configuration, it is copied to the */etc/nginx/sites-available* directory, and a symbolic link file to it is created in the */etc/nginx/sites-enabled* directory. The content of the *example.domain* is as follows:

```
server {
    listen 80;
    listen [::]:80;
    server_name example.domain www.example.domain;

    root /var/www/html;

    # Add index.php to the list if you are using PHP
    index index.html index.htm index.nginx-debian.html;

    location / {
        return 301 $scheme://www.example.domain/
        home$request_uri;
    }

    location /home {
        try_files $uri $uri/ =404;
    }
}
```

The Nginx Web server is then started to serve the newly copied content. The Ansible playbook for copying and serving the static files can be invoked as follows:

```
$ ansible-playbook -i inventory/kvm/inventory playbooks/
configuration/nginx.yml --tags static -vv -K
```

You can now refresh the <http://192.168.122.140> page on your local host to see your static Web content.

Goaccess

The Goaccess interactive, real-time Web log analyser is available in the Debian package repository. After updating the software APT repository, Goaccess is installed. The configuration format to parse the Nginx logs is made available in the *goaccessrc* file. Its contents are as follows:

```
log-format %h %A[%d:%t %A] "%r" %s %b %R" %u" %T %A
time-format %H:%M:%S
date-format %d/%b/%Y
```

The above code is copied to *~/goaccessrc*. The Ansible playbook to install Goaccess is given below:

```
- name: Install goaccess
  hosts: debian
  become: yes
```